

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Name
Abgabedatum: TT.MM.JJJJ

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Das Vorwort wird in der Endfassung bei Bedarf um persönliche Danksagungen und organisatorische Hinweise ergänzt.

Contents

Vorwort	ii
List of Figures	v
List of Tables	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung der Arbeit	1
1.3 Thematische Ausrichtung der Arbeit	2
1.4 Vorgehensweise und Aufbau der Arbeit	2
1.5 Leitgedanke der Arbeit	3
2 Grundlagen	4
2.1 Betriebssysteme und Prozessverwaltung	4
2.2 CPU-Scheduling in Betriebssystemen	4
2.3 Präemptive Scheduling-Verfahren	4
2.4 Überblick über die betrachteten Algorithmen	5
2.5 Kriterien zur Bewertung von Scheduling-Verfahren	5
3 Systemarchitektur und Komponentenmodell	7
3.1 Ziel und Bedeutung der Architektur	7
3.2 Architekturprinzipien	7
3.3 Komponentenmodell der Anwendung	8
3.4 Vektorgrafik: Komponentenmodell der Anwendung	8
3.5 Beschreibung der Hauptkomponenten	8
3.6 Beziehung zwischen den Komponenten	10
3.7 Erklärende Darstellung des Komponentenmodells	10
3.8 Zusammenfassung	11
4 Grundlagen der Mensch-Computer-Interaktion	12
4.1 Ziele der Mensch-Computer-Interaktion	12
4.2 Usability und Nutzerfreundlichkeit	12
4.3 Interaktionsgestaltung für Lernanwendungen	12
4.4 Rückmeldung, Transparenz und Verständlichkeit	13
4.5 Anforderungen an die Benutzungsoberfläche des Visualizers	13
5 Anforderungen und Konzeption des Scheduling-Visualizers	15
5.1 Zielgruppe und Einsatzkontext	15
5.2 Fachliche Anforderungen	15
5.3 Didaktische Anforderungen	16

5.4	Funktionale Anforderungen	16
5.5	Nicht-funktionale Anforderungen	17
5.6	Software-Engineering-Qualitätsziele	17
5.7	Konzeption der Simulationslogik	18
5.8	Konzeption der Visualisierung	18
5.9	Konzeption der Benutzungsoberfläche	18
6	Implementierung	20
6.1	Technische Grundlagen und verwendete Technologien	20
6.2	Architektur der Anwendung	20
6.3	Modellierung der Scheduling-Verfahren	21
6.4	Umsetzung der Simulation	21
6.5	Umsetzung der Visualisierung	22
6.6	Umsetzung der Interaktionsmechanismen	26
6.7	Besondere technische Herausforderungen	27
6.8	Teststrategie und Qualitätssicherung	28
7	Ergebnis und Produktpräsentation	29
7.1	Entwickeltes Endprodukt	29
7.2	Produktüberblick und Nutzerfluss	29
7.3	Produktpräsentation mit Screenshots	30
7.4	Umgesetzte Funktionalität	30
7.5	Ergebnisbetrachtung	34
8	Fazit und Ausblick	35
8.1	Zusammenfassung der Ergebnisse	35
8.2	Reflexion über die gestellten Anforderungen	35
8.3	Ergebnis und praktische Wirkung	36
8.4	Ausblick auf Weiterentwicklung	36
9	Anhang	39

List of Figures

3.1	Komponentenmodell der Scheduling-Visualisierungsanwendung. Die Benutzeroberfläche nutzt die fachliche Simulationslogik; Ergebnisse werden anschließend in Visualisierungs- und Vergleichskomponenten dargestellt. . .	9
5.1	Szenario-Generator zur Definition von Task-Daten und Simulationsparametern.	19
6.1	Ausgangszustand ohne Szenario und Algorithmus.	22
6.2	Zustand nach dem Laden eines Szenarios ohne angewendeten Algorithmus.	23
6.3	Fokusansicht mit Gantt-Zeitachse, Stack-Visualisierung und Kennzahlen. . .	23
6.4	Detailansicht der Gantt-Zeitachse	24
6.5	Detailansicht der Stack-Visualisierung mit aktivem und wartenden Task. . .	24
6.6	Detailansicht der berechneten Kennzahlen eines Simulationslaufs.	25
6.7	Gesamtübersicht der Vergleichsansicht mit Mini-Gantts und Bewertungsmatrix.	25
6.8	Vergleichsübersicht mit nebeneinander dargestellten Mini-Gantts mehrerer Runs.	26
6.9	Bewertungsmatrix zur gewichteten Einordnung mehrerer Simulationsläufe. .	26
7.1	Startansicht der Anwendung mit leerem Zustand und erklärender Ausgangslage. Der Nutzer startet mit der Definition eines Szenarios und der Auswahl eines Scheduling-Verfahrens.	30
7.2	Ladezustand eines konfigurierten Szenarios. Die Task-Daten sind sichtbar, und die Simulation kann mit einer klaren Ablaufsteuerung initialisiert werden.	31
7.3	Fokusansicht der Simulation mit Gantt-Zeitachse, aktiver Task-Darstellung und Kennzahlbereich. Diese Ansicht zeigt den Ablauf des gewählten Scheduling-Verfahrens in einer verständlichen, zeitlich strukturierten Form.	31
7.4	Detailansicht der Gantt-Darstellung. Einzelne Ausführungssegmente, Präemptionen und Task-Wechsel werden als konkrete Zeitabschnitte sichtbar gemacht.	32
7.5	Darstellung des Task-Status in der Warteschlangenansicht. Der aktuelle Zustand und die Warteliste werden gleichzeitig präsentiert, wodurch die Auswahlentscheidung des Algorithmus nachvollziehbar wird.	32
7.6	Kennzahlenbereich mit Warte-, Durchlauf- und Reaktionszeiten. Die Metriken geben eine kompakte fachliche Bewertung des Simulationslaufs wieder.	33
7.7	Vergleichsübersicht mit mehreren Läufen unter identischem Szenario. Die Darstellung ermöglicht den praktischen Vergleich verschiedener Scheduling-Verfahren anhand des selben Dateninputs.	33

List of Tables

Abkürzungsverzeichnis

CPU Central Processing Unit

HCI Human-Computer Interaction

MLFQ Multilevel Feedback Queue

RR Round Robin

SRTF Shortest Remaining Time First

1 Einleitung

1.1 Motivation und Problemstellung

Scheduling-Verfahren gehören zu den zentralen Konzepten moderner Betriebssysteme. Sie bestimmen, welcher Prozess zu welchem Zeitpunkt CPU-Zeit erhält und beeinflussen unmittelbar Reaktionsfähigkeit, Fairness und Auslastung eines Systems. Obwohl diese Verfahren in Lehrbüchern und Vorlesungen theoretisch sauber beschrieben werden, fällt es Studierenden häufig schwer, ihren zeitlichen Ablauf in konkreten Situationen zu verstehen [8, 10].

Gerade präemptive Verfahren sind für Lernende anspruchsvoll, weil sie viele Zustandswechsel, Unterbrechungen und Neuplanungen erzeugen. Die Entscheidungslogik bleibt bei reinen Textbeschreibungen und statischen Beispielen oft abstrakt. Das Problem zeigt sich besonders beim Vergleich verschiedener Verfahren unter identischen Eingaben: Die fachlichen Unterschiede sind zwar theoretisch bekannt, aber in ihrer unmittelbaren Wirkung auf Ablauf, Zeitverlauf und Fairness nicht immer intuitiv nachvollziehbar.

Vor diesem Hintergrund wurde eine interaktive Webanwendung entwickelt, die Scheduling-Verfahren als sichtbaren, steuerbaren Ablauf modelliert. Ziel ist nicht die Beantwortung einer rein wissenschaftlichen Forschungsfrage, sondern die Erstellung eines nutzbaren, didaktischen Werkzeugs, das technische Zusammenhänge transparent macht und Studierenden einen praktischen Zugang zu den Grundlagen des CPU-Scheduling eröffnet.

1.2 Zielsetzung der Arbeit

Die vorliegende Arbeit verfolgt die Konzeption und Umsetzung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Das Ziel besteht darin, eine Anwendung zu entwickeln, die:

- ausgewählte Scheduling-Verfahren fachlich korrekt simuliert,
- ihre zeitlichen Abläufe verständlich visualisiert,
- verschiedene Verfahren anhand identischer Szenarien vergleichbar macht,
- und als didaktisches Werkzeug im Lehrkontext nutzbar ist.

Der Fokus liegt damit auf der praktischen Umsetzung eines verständlichen und erweiterbaren

Produkts. Die Arbeit verbindet fachliche Grundlagen aus der Betriebssystemtheorie mit Prinzipien aus Software Engineering und Human-Computer Interaction. Dabei steht die nachvollziehbare Architektur, die klare Interaktion und die unmittelbare Produktwirkung im Vordergrund.

1.3 Thematische Ausrichtung der Arbeit

Die Arbeit ist bewusst nicht als klassische empirische Untersuchung konzipiert, sondern als engineering-orientierte Produktdokumentation. Der Schwerpunkt liegt auf der Gestaltung einer Anwendung, die im Studienkontext einen Mehrwert bietet und zugleich technische, didaktische und gestalterische Entscheidungen nachvollziehbar dokumentiert.

Diese Ausrichtung ist für den Studiengang geeignet, weil sie die Verbindung zwischen theoretischem Wissen und praktischer Umsetzung sichtbar macht. Die fachlichen Grundlagen aus der Betriebssystemlehre werden mit einer strukturierten Architektur, benutzerfreundlicher Oberfläche und konkreten Produktbeispielen verknüpft. Dadurch entsteht eine Bachelorarbeit, die nicht nur theoretisch fundiert ist, sondern auch die Entwicklung eines nutzbaren Softwaresystems als zentrale Leistung hervorhebt.

1.4 Vorgehensweise und Aufbau der Arbeit

Die Arbeit folgt einem engineering-orientierten Vorgehen:

1. Analyse der fachlichen Grundlagen zu CPU-Scheduling und relevanten Messgrößen,
2. Ableitung von Anforderungen an eine didaktische Visualisierungsanwendung,
3. Konzeption der Systemarchitektur und der Komponentenstruktur,
4. technische Umsetzung der Webanwendung,
5. produktorientierte Darstellung der Ergebnisse mit Screenshots und systematischer Erklärung.

Der Aufbau der Bachelorarbeit ist entsprechend auf die Entwicklung und Präsentation des Produkts ausgerichtet. Kapitel 2 behandelt die fachlichen Grundlagen. Kapitel 3 und Kapitel 4 legen die Anforderungen an die Architektur und die HCI-gestützte Gestaltung fest. Kapitel 5 beschreibt die konkrete Konzeption und den Entwurf der Anwendung. Kapitel 6 dokumentiert die technische Implementierung. Kapitel 7 stellt das Ergebnis mit der Produktpräsentation und der Visualisierung der wichtigsten Funktionen dar. Kapitel 8 schließt mit Reflexion, Grenzen und Ausblick.

1.5 Leitgedanke der Arbeit

Der zentrale Gedanke dieser Arbeit lautet:

Praktische Verständlichkeit entsteht nicht nur durch fachliche Korrektheit, sondern auch durch eine gut strukturierte Architektur, klare Interaktion und eine verständliche Darstellung des zeitlichen Ablaufs.

Diese Perspektive bildet den Kern der Arbeit. Sie macht deutlich, dass die Simulation von Scheduling-Verfahren nicht nur eine fachliche Aufgabe ist, sondern auch ein Design- und Architekturproblem mit unmittelbaren Auswirkungen auf Nutzbarkeit und Lernwirkung.

2 Grundlagen

Dieses Kapitel beschreibt die theoretischen Grundlagen, auf denen Konzeption, Implementierung und Evaluation der Anwendung aufbauen.

2.1 Betriebssysteme und Prozessverwaltung

Betriebssysteme koordinieren den Zugriff auf Hardware-Ressourcen und stellen Ausführungsumgebungen für Programme bereit. Eine Kernaufgabe ist dabei die Verwaltung konkurrierender Prozesse, die um CPU-Zeit, Speicher und weitere Ressourcen konkurrieren [8, 10].

Prozesse durchlaufen typischerweise mehrere Zustände, etwa bereit, laufend und blockiert. Das Betriebssystem steuert Übergänge zwischen diesen Zuständen und bestimmt durch Scheduling-Entscheidungen, welcher bereite Prozess als nächstes ausgeführt wird. Diese Entscheidungen wirken sich direkt auf wahrgenommene Systemreaktion und Gesamtdurchlauf aus.

2.2 CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie für die Vergabe von Prozessorzeit an bereite Prozesse. Ziel ist ein sinnvoller Ausgleich zwischen konkurrierenden Kriterien wie kurzer Reaktionszeit, niedriger Wartezeit, hoher Auslastung und fairer Behandlung [7, 8].

In Mehrprogrammsystemen ist Scheduling eine kontinuierliche Entscheidung unter Unsicherheit, da Ankunftszeitpunkte und Prozessverhalten variieren. Deshalb sind Scheduling-Verfahren typischerweise als Heuristiken formuliert, deren Eignung vom Lastprofil und vom Systemziel abhängt.

2.3 Präemptive Scheduling-Verfahren

Präemptive Verfahren erlauben es dem Betriebssystem, einen laufenden Prozess aktiv zu unterbrechen und einen anderen Prozess einzuplanen. Dies verbessert in vielen Kontexten die Reaktionsfähigkeit, verursacht jedoch zusätzlichen Verwaltungsaufwand durch Kontextwechsel [10, 8].

Der zentrale Unterschied zu nicht-präemptiven Strategien besteht darin, dass Entscheidungen nicht nur bei Abschluss oder Blockierung eines Prozesses getroffen werden, sondern auch während seiner Ausführung. Dadurch steigt die Steuerbarkeit, gleichzeitig aber auch die Komplexität von Analyse und Visualisierung [10].

2.4 Überblick über die betrachteten Algorithmen

2.4.1 Last Come First Served

Last Come First Served (LCFS) priorisiert neu eingetroffene Prozesse. In der präemptiven Variante kann ein neu ankommender Prozess den aktuell laufenden Prozess sofort verdrängen. Das Verfahren ist anschaulich, kann aber unter ungünstigen Lastprofilen zu langen Wartezeiten früher eingetroffener Prozesse führen.

2.4.2 Round Robin

Round Robin (RR) weist jedem Prozess ein Zeitquantum zu. Nach Ablauf des Quantums wird der Prozess, falls noch nicht abgeschlossen, wieder in die Bereitwarteschlange eingeordnet. RR gilt als faires Grundverfahren für zeitscheibenbasierte Systeme, da alle Prozesse regelmässig CPU-Zeit erhalten [8].

2.4.3 Strict Priority

Strict Priority wählt stets den Prozess mit der höchsten Priorität aus. Trifft ein noch höher priorisierter Prozess ein, kann eine sofortige Präemption stattfinden. Vorteilhaft ist die gezielte Bevorzugung wichtiger Aufgaben; problematisch ist jedoch das Risiko von Starvation für niedrig priorisierte Prozesse [10].

2.4.4 Multilevel Feedback Queue

Multilevel Feedback Queue (MLFQ) kombiniert mehrere Warteschlangenebenen mit unterschiedlichen Prioritäten und Zeitquanten. Prozesse können in niedrigere Ebenen verschoben werden, wenn sie CPU-intensiv sind, während kurz laufende oder interaktive Prozesse in höheren Ebenen profitieren. Dadurch entsteht ein adaptives Verhalten, das auf heterogene Lastprofile reagieren kann [8, 5].

2.5 Kriterien zur Bewertung von Scheduling-Verfahren

Die Bewertung von Scheduling-Verfahren erfolgt über klar definierte Kennzahlen. Diese sind entscheidend, um Algorithmen nicht nur qualitativ, sondern auch quantitativ zu

vergleichen [7].

2.5.1 Wartezeit

Die Wartezeit beschreibt die Zeitspanne, die ein Prozess im Zustand „bereit“ verbringt, ohne ausgeführt zu werden. Niedrige durchschnittliche Wartezeiten sind insbesondere in Systemen mit vielen kurzen Aufgaben wünschenswert.

2.5.2 Durchlaufzeit

Die Durchlaufzeit (Turnaround Time) ist die Zeit vom Eintreffen eines Prozesses bis zu dessen Abschluss. Sie bildet den Gesamtablauf aus Sicht eines Auftrags ab und ist eine zentrale Effizienzmetrik für batch-orientierte Lastprofile.

2.5.3 Reaktionszeit

Die Reaktionszeit beschreibt die Zeit vom Eintreffen eines Prozesses bis zu seiner ersten CPU-Zuteilung. Diese Kennzahl ist besonders relevant für interaktive Systeme, in denen frühes Feedback wichtiger sein kann als minimale Gesamtdurchlaufzeit [8, 10].

Ergänzend können Kontextwechselhäufigkeit, Präemptionsanzahl, Fairnessmasse und CPU-Leerlaufanteile betrachtet werden, um Zielkonflikte zwischen Responsivität, Stabilität und Ressourceneffizienz sichtbar zu machen.

3 Systemarchitektur und Komponentenmodell

3.1 Ziel und Bedeutung der Architektur

Die Architektur der Anwendung ist ein zentraler Bestandteil des Projekts, weil sie den fachlichen Kern von der visuellen Darstellung und der Interaktion sauber trennt. Eine gut strukturierte Architektur ist für ein Lehrwerkzeug besonders wichtig, da sie nicht nur Wartbarkeit und Erweiterbarkeit sichert, sondern auch die Nachvollziehbarkeit der Scheduling-Entscheidungen erhöht.

Das System wurde deshalb so konzipiert, dass die fachliche Logik der Scheduling-Algorithmen von der Benutzeroberfläche und den Anzeigekomponenten entkoppelt ist. Dadurch bleibt der Simulationskern klar definiert, reproduzierbar und testbar. Die Oberfläche kann anschließend die erzeugten Zustände, Ereignisse und Kennzahlen darstellen, ohne die eigentliche Berechnung zu beeinflussen.

3.2 Architekturprinzipien

Die Anwendung folgt einer komponentenbasierten Struktur mit klaren Verantwortlichkeiten. Die wichtigsten Prinzipien sind:

- **Trennung von Logik und Darstellung:** Die Berechnung des Ablaufs liegt in der Simulationslogik; die UI zeigt nur die resultierenden Zustände an.
- **Deterministische Verarbeitung:** Gleiche Eingaben erzeugen dieselben Laufdaten und dieselben Berechnungen.
- **Modularität:** Die Struktur wird in fachlich getrennte Bereiche aufgeteilt, damit einzelne Teile unabhängig entwickelt, getestet und erweitert werden können.
- **Nachvollziehbarkeit:** Jede Darstellung lässt sich auf einen konkreten Simulation-zustand zurückführen.

Diese Architekturentscheidungen sind für den Studiengang besonders relevant, weil sie typische Software-Engineering-Prinzipien sichtbar machen: Lose Kopplung, klare Zuständigkeiten, Wiederverwendbarkeit und Testbarkeit.

3.3 Komponentenmodell der Anwendung

Das Komponentenmodell beschreibt die wichtigsten Bausteine der Anwendung und deren Beziehungen zueinander. Die Struktur lässt sich in vier Ebenen gliedern:

- **Benutzerschnittstelle:** Szenario-Editor, Steuerungselemente, Ergebnisansichten und Vergleichsbereiche.
- **Anwendungslogik:** Verwaltung von Prozessdaten, Laufkonfigurationen und Zustandswechseln.
- **Simulationskern:** Berechnung der Scheduling-Entscheidungen, Ereignisse und Kennzahlen.
- **Visualisierung:** Zeitachsen, Gantt-Darstellung, Kennzahlen-Panel und Vergleichsübersicht.

Diese Abgrenzung ist nicht nur technisch sinnvoll, sondern auch didaktisch relevant: Nutzerinnen und Nutzer sehen die fachliche Logik nicht als Black Box, sondern als klar strukturierte, nachvollziehbare Verarbeitungskette. Das erhöht sowohl das Verständnis für das System als auch das Vertrauen in die dargestellten Ergebnisse.

3.4 Vektorgrafik: Komponentenmodell der Anwendung

3.5 Beschreibung der Hauptkomponenten

3.5.1 Szenario- und Konfigurationskomponente

Diese Komponente verwaltet die Eingabedaten eines Laufes. Sie enthält Prozessinformationen wie Ankunftszeit, Burst-Zeit, Priorität und Laufparameter. Über sie werden neue Szenarien erzeugt und bestehende Konfigurationen angepasst.

3.5.2 Simulationskern

Der Simulationskern bildet das fachliche Zentrum der Anwendung. Er verarbeitet die Prozesse nach den Regeln des ausgewählten Scheduling-Verfahrens und erzeugt den zeitlichen Ablauf, den Ereignislog und die relevanten Kennzahlen. Innerhalb dieser Schicht erfolgen die eigentlichen Entscheidungen über Auswahl, Präemption und Neuordnung der Prozesse.

3.5.3 Wiedergabe- und Steuerungslogik

Die Wiedergabe- und Steuerungslogik bestimmt, welche Zustände des Simulationsverlaufs zu welchem Zeitpunkt in der Oberfläche sichtbar sind. Sie steuert Start, Pause,

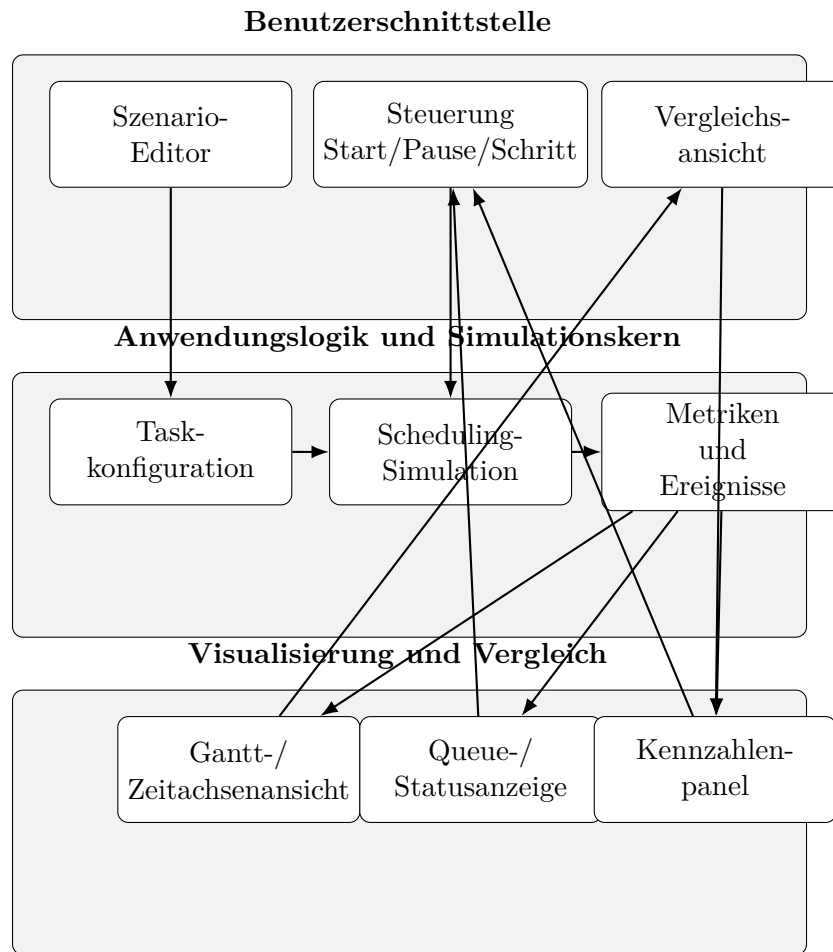


Figure 3.1: Komponentenmodell der Scheduling-Visualisierungsanwendung. Die Benutzeroberfläche nutzt die fachliche Simulationslogik; Ergebnisse werden anschließend in Visualisierungs- und Vergleichskomponenten dargestellt.

Schrittmodus und Geschwindigkeit, ohne die fachliche Berechnung erneut zu initiieren.

3.5.4 Visualisierungs- und Vergleichskomponenten

Diese Komponenten rendern den Ablauf als Gantt-Diagramm, ergänzen Kennzahlen und erlauben die Gegenüberstellung mehrerer Läufe. Die Darstellung ist somit nicht die Ursache der Berechnung, sondern die sichtbare Ableitung eines bereits bestehenden Simulationszustands.

3.6 Beziehung zwischen den Komponenten

Die einzelnen Module arbeiten als Flussmodell zusammen: Ein Nutzer konfiguriert ein Szenario, der Simulationskern berechnet den Ablauf, die Wiedergabe-Logik zeigt den richtigen Zeitstand an und die Visualisierungs-Komponenten präsentieren die finalen Ergebnisse. Diese Struktur sorgt dafür, dass Änderungen an einem Bereich nicht unmittelbar die fachliche Logik oder die Darstellung anderer Bereiche zerstören.

Diese Form der Entkopplung ist in Software Engineering ein wichtiges Qualitätsmerkmal, weil sie die Anwendung modular und erweiterbar macht. Insbesondere für ein Lehrprojekt ist das sinnvoll: neue Scheduling-Verfahren, neue Kennzahlen oder zusätzliche Visualisierungsansichten lassen sich mit vergleichsweise geringem Aufwand ergänzen.

3.7 Erklärende Darstellung des Komponentenmodells

Das Komponentenmodell wird in der Vektorgrafik als Architekturübersicht dargestellt. Die Abbildung zeigt die zentrale Rolle des Simulationskerns und die Verbindung zwischen Eingabe, Verarbeitung und Visualisierung. Diese Darstellung ist wichtig, weil sie das technische Verständnis der Anwendung unterstützt und zugleich die fachliche Struktur der Lösung transparent macht.

Die Abbildung verdeutlicht drei zentrale Aspekte:

- Die Benutzeroberfläche ist nicht der eigentliche Rechenort der Scheduling-Logik.
- Die fachliche Berechnung wird in einem klar abgegrenzten Kern modelliert.
- Die Visualisierung ist eine nachgelagerte Schicht, die den Ablauf für den Nutzer verständlich macht.

Diese Architektur ist eine zentrale Grundlage der Arbeit, da sie die technische Qualität und den professionellen Entwurf des Systems belegt. Sie zeigt, dass es sich bei der Anwendung nicht nur um ein einfaches Demo-Tool handelt, sondern um eine nachvollziehbar designte, erweiterbare und modular aufgebaute Software.

3.8 Zusammenfassung

Das vorgestellte Komponentenmodell macht den Kern der Arbeit sichtbar: Die Anwendung verbindet fachliche Korrektheit mit klarer Softwarearchitektur und verständlicher Darstellung. Die Systemstruktur ist damit nicht nur für die technische Umsetzung relevant, sondern auch für die didaktische Qualität der Anwendung. Ein gut gegliedertes System reduziert Komplexität, verbessert die Wartbarkeit und ermöglicht eine verständliche Präsentation der Ideen hinter CPU-Scheduling.

4 Grundlagen der Mensch-Computer-Interaktion

4.1 Ziele der Mensch-Computer-Interaktion

Mensch-Computer-Interaktion beschreibt die Gestaltung digitaler Systeme so, dass sie für Menschen verständlich, effizient und verlässlich nutzbar sind. Bei einer Lernanwendung wie dem Scheduling-Visualizer steht nicht allein die funktionale Korrektheit im Vordergrund, sondern auch die Qualität der Interaktion.

Ziel ist eine Oberfläche, die Nutzerinnen und Nutzer sicher durch den Lernprozess führt: Eingaben sollen nachvollziehbar sein, Zustandsänderungen klar rückgemeldet werden, und Bedienhandlungen müssen ohne unnötige Hürden ausführbar bleiben.

4.2 Usability und Nutzerfreundlichkeit

Usability umfasst die Frage, wie wirksam, effizient und zufriedenstellend ein System in einem konkreten Nutzungskontext bedient werden kann. Für den Scheduling-Visualizer bedeutet das unter anderem, dass Interaktionen verständlich, effizient und fehlertolerant gestaltet sein müssen. Für die spätere Bewertung kann dabei auf etablierte Verfahren wie die System Usability Scale zurückgegriffen werden [2].

- Prozesse und Parameter sollen mit möglichst wenigen Schritten erfasst werden können,
- Simulationszustände müssen eindeutig erkennbar sein,
- und Fehlbedienungen sollen durch klares Feedback schnell korrigierbar bleiben.

Eine hohe Nutzerfreundlichkeit kann dazu beitragen, dass kognitive Ressourcen nicht in Bedienproblemen gebunden werden, sondern für das fachliche Verständnis verfügbar bleiben.

4.3 Interaktionsgestaltung für Lernanwendungen

Interaktionsgestaltung im Lernkontext folgt dem Prinzip der didaktischen Unterstützung: Das Interface soll Lernziele erleichtern, nicht verdecken. Im vorliegenden Fall betrifft das

insbesondere die Steuerung des Zeitverlaufs und den Vergleich verschiedener Scheduling-Strategien.

Sinnvolle Interaktion entsteht dabei durch eine klare Aufgabenteilung zwischen Eingabe, Steuerung und Ergebnisdarstellung. Wenn diese Bereiche konsistent strukturiert sind, können Nutzende schneller zwischen Experimentieren und Interpretieren wechseln.

4.4 Rückmeldung, Transparenz und Verständlichkeit

Ein zentrales HCI-Prinzip ist die Sichtbarkeit des Systemzustands. Im Scheduling-Visualizer müssen daher wesentliche Informationen permanent oder mit minimalem Aufwand abrufbar sein. Gleichzeitig sind Anforderungen an Barrierefreiheit früh in die Interaktionsgestaltung zu integrieren, damit zentrale Funktionen für möglichst viele Nutzende zugänglich bleiben [14]:

- aktueller Simulationszeitpunkt,
- gerade laufender Prozess,
- Warteschlangen- und Prioritätszustände,
- wichtige Ereignisse wie Präemptionen und Kontextwechsel.

Rückmeldungen sollen nicht nur korrekt, sondern auch semantisch klar formuliert sein. Das reduziert Interpretationsfehler und erhöht die Nachvollziehbarkeit von Algorithmusentscheidungen.

4.5 Anforderungen an die Benutzungsoberfläche des Visualizers

Auf Basis der HCI-Analyse ergeben sich konkrete Anforderungen an die Benutzungsoberfläche. Die folgenden Punkte konkretisieren diese Anforderungen:

- **Konsistente Navigations- und Fokusführung:** Interaktive Elemente müssen in logischer Reihenfolge erreichbar sein.
- **Barrierearme Interaktion:** Tastaturbedienung, semantische Beschriftungen und ausreichende Kontraste sind notwendig.
- **Robuste Dialoginteraktion:** Modale Dialoge brauchen klares Fokusmanagement sowie nachvollziehbares Schliessen.
- **Eindeutige Statuskommunikation:** Der Unterschied zwischen laufender, pausierter und abgeschlossener Simulation muss sofort erkennbar sein.
- **Fehlertoleranz:** Nutzende sollen Eingaben korrigieren können, ohne den gesamten Arbeitskontext zu verlieren.

Diese Anforderungen greifen die vorhandenen Erkenntnisse zur Interaktionsqualität auf

und bilden die Grundlage für die anschliessende Konzeption und Implementierung der Anwendung. Damit werden didaktische Zielsetzung und HCI-Qualitätskriterien systematisch zusammengeführt [4, 14].

5 Anforderungen und Konzeption des Scheduling-Visualizers

Dieses Kapitel definiert die fachlichen, didaktischen und technischen Anforderungen der Anwendung und leitet daraus das konzeptionelle Lösungsdesign ab. Im Mittelpunkt steht die Frage, wie präemptive Scheduling-Verfahren fachlich korrekt, didaktisch nachvollziehbar und mit hoher Interaktionsqualität visualisiert werden können. Die konzeptionellen Entscheidungen orientieren sich dabei an etablierten Scheduling-Grundlagen [8, 10, 7], an Erkenntnissen aus der Algorithmusvisualisierung [4, 6] sowie an Anforderungen der Usability und Accessibility [2, 14].

5.1 Zielgruppe und Einsatzkontext

Primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll insbesondere in Situationen unterstützen, in denen formale Algorithmusbeschreibungen allein nicht ausreichen, um den zeitlichen Ablauf präemptiver Entscheidungen zu verstehen.

Der Einsatzkontext umfasst sowohl Lehrveranstaltungen mit geführter Demonstration als auch das selbstständige Lernen. Daraus folgen zwei zentrale Rahmenbedingungen: Erstens muss die Anwendung reproduzierbare Ergebnisse liefern, damit Beobachtungen zwischen verschiedenen Nutzenden vergleichbar bleiben. Zweitens braucht sie eine flexible Interaktion, damit sowohl exploratives Ausprobieren als auch gezieltes Nachstellen vorgegebener Szenarien möglich ist.

5.2 Fachliche Anforderungen

Aus fachlicher Sicht muss der Visualizer die betrachteten Verfahren konsistent und korrekt modellieren. Dazu gehören insbesondere LCFS, Round Robin, Strict Priority und MLFQ. Relevante Kernaspekte sind Ankunftszeit, Burst-Zeit, Priorität, Zeitquantum, Präemption und Kontextwechsel [8, 10].

Zusätzlich müssen die wichtigsten Vergleichsmetriken nachvollziehbar berechnet werden. Dazu zählen Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechselhäufigkeit und weitere Kenngrößen zur Bewertung von Fairness und Ressourcennutzung [7]. Die konzeptionellen

tionelle Anforderung lautet daher, dass Simulationslogik und Metrikberechnung auf derselben Zustandsbasis arbeiten und keine voneinander getrennten Berechnungspfade verwenden.

5.3 Didaktische Anforderungen

Didaktisch soll die Anwendung nicht nur Ergebnisse anzeigen, sondern den Entscheidungsprozess der Algorithmen sichtbar machen. Lernende sollen erkennen können, warum ein Task zu einem bestimmten Zeitpunkt ausgewählt, unterbrochen oder erneut eingeplant wird. Dieser Anspruch folgt aus Forschung zur Algorithmusvisualisierung, die aktive Interaktion und vergleichende Betrachtung als potenziell lernunterstützende Gestaltungsmerkmale beschreibt [4, 6, 9].

Die Konzeption priorisiert deshalb drei didaktische Leitprinzipien:

- **Transparenz der Zustandsübergänge:** Ereignisse wie Start, Präemption und Abschluss müssen nachvollziehbar dargestellt werden.
- **Vergleichbarkeit:** Mehrere Algorithmen sollen unter identischen Eingabedaten gegenübergestellt werden können.
- **Kontrollierbares Lerntempo:** Schrittweises und automatisches Abspielen sollen gleichwertig unterstützt werden.

5.4 Funktionale Anforderungen

Funktional muss die Anwendung den gesamten Ablauf von der Szenariodefinition bis zur Ergebnisinterpretation abdecken. Daraus ergeben sich folgende Kernfunktionen:

- Erfassung und Bearbeitung von Task-Mengen mit relevanten Attributen.
- Unterstützung bei der Szenariogenerierung durch vordefinierte Basis-Szenarien und eine interaktive Eingabemaske.
- Auswahl eines Scheduling-Verfahrens und seiner Parameter über einen klar strukturierten Dialog.
- Start, Pause, Schrittmodus und Zeitsprung innerhalb der Simulation.
- Visualisierung der Ausführungssegmente auf einer Zeitachse.
- Ausgabe vergleichbarer Metriken pro Lauf.
- Gegenüberstellung mehrerer Läufe auf identischem Szenario mit Bewertungs- und Vergleichslogik.

Konzeptionell wird damit ein durchgängiger Workflow vom Eingabemodell zur Auswertung abgebildet. Jede Funktion muss auf denselben Simulationsdaten aufbauen, um Inkonsis-

tenzen zwischen Anzeige und Metrikberechnung zu vermeiden. Die zusätzliche Szenariogenerierung und die Vergleichsübersicht erhöhen dabei den praktischen Nutzen der Anwendung, da sie den Nutzungsfluss von der ersten Definition bis zur fachlichen Bewertung deutlich besser unterstützen.

5.5 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen betreffen vor allem Nachvollziehbarkeit, Robustheit und Nutzbarkeit. Die Anwendung soll deterministisch arbeiten, damit identische Eingaben zu identischen Ergebnissen führen. Dies ist für didaktische Vergleichbarkeit und wissenschaftliche Reproduzierbarkeit zentral [1].

Darüber hinaus gelten Anforderungen an Performance und Bedienqualität: Interaktionen müssen unmittelbar rückmelden, die Darstellung muss auch bei grösseren Szenarien stabil bleiben, und zentrale Funktionen müssen ohne Maus nutzbar sein. Anforderungen an semantische Benennung, Fokusführung und Kontrast orientieren sich an etablierten Accessibility-Kriterien [14]. Für die spätere Nutzerbewertung wird ein standardisiertes Usability-Instrument wie SUS als geeignet betrachtet [2].

5.6 Software-Engineering-Qualitätsziele

Neben den fachlichen und didaktischen Anforderungen spielen in einer engineering-orientierten Arbeit auch strukturelle Qualitätsziele eine entscheidende Rolle. Die Anwendung wurde deshalb mit Blick auf Wartbarkeit, Erweiterbarkeit und Testbarkeit entworfen. Diese Ziele sind für ein Lehrprojekt besonders wichtig, weil sie zeigen, dass das System nicht nur als Demo, sondern als nachvollziehbar entwickeltes Softwaresystem konzipiert wurde.

Ein zentrales Qualitätsziel ist die klare Trennung von Verantwortlichkeiten. Die fachliche Logik der Scheduling-Verfahren, die Zustandsverwaltung der Simulation und die Darstellung im Frontend werden bewusst voneinander abgegrenzt. Dadurch wird die Kopplung zwischen den Komponenten reduziert und Änderungen an einem Bereich wirken sich nicht unmittelbar auf die übrigen Schichten aus. Dieses Vorgehen entspricht den grundlegenden Prinzipien des Software Engineering und erhöht die langfristige Qualität des Systems.

Ein weiteres Qualitätsmerkmal ist die Reproduzierbarkeit. Die Simulation soll bei identischen Eingabedaten immer denselben Ablauf und dieselben Kennzahlen erzeugen. Dieses Verhalten ist für den didaktischen Einsatz relevant, da Lernende auf dieselben Ergebnisse vertrauen und die Auswirkungen von Parametern oder Verfahrensentscheidungen direkt vergleichen können. Reproduzierbarkeit ist damit nicht nur ein technisches, sondern auch ein didaktisches Qualitätsmerkmal.

Zusätzlich wird die Anwendung als erweiterbare Architektur modelliert. Neue Scheduling-

Verfahren, zusätzliche Kennzahlen oder alternative Visualisierungsebenen lassen sich an die bestehende Struktur anfügen, ohne die gesamte Simulationsarchitektur neu zu entwerfen. Diese Flexibilität ist in engineering-Projekten ein zentraler Erfolgsfaktor, weil sie das System über einen kurzen Prototypen hinaus zu einer stabilen Grundlage für weitere Iterationen macht.

5.7 Konzeption der Simulationslogik

Die Simulationslogik wird als deterministischer Kern konzipiert, der diskrete Zeitschritte verarbeitet und Ereignisse in wohldefinierter Reihenfolge erzeugt. Die zentrale Entscheidung besteht darin, Simulationszustand, Ereignisprotokoll, Zeitachsen-Segmente und Metriken gemeinsam aus einem einheitlichen Zustandsmodell abzuleiten. Dadurch bleiben alle Ausgaben konsistent.

Konzeptionell werden dabei zwei Anforderungen verbunden: erstens fachliche Korrektheit der Scheduling-Regeln, zweitens auswertbare Verlaufsdaten für Visualisierung und Evaluation. Das Ereignismodell muss daher nicht nur Endergebnisse liefern, sondern auch Zwischenschritte explizit protokollieren, damit Präemptionen und Kontextwechsel analytisch nachvollzogen werden können [8, 7].

5.8 Konzeption der Visualisierung

Als primäres Visualisierungsmedium wird ein Gantt-artiger Zeitverlauf eingesetzt. Diese Form ist geeignet, da sie parallel Informationen über Task-Dauer, Unterbrechungen und Wartephasen abbildet und damit den Vergleich unterschiedlicher Strategien erleichtert.

Die Visualisierung wird konzeptionell als Ableitung des Simulationskerns verstanden, nicht als eigene Logikinstanz. Das bedeutet: Die Darstellung interpretiert vorhandene Zustands- und Segmentdaten, trifft aber keine eigenständigen Scheduling-Entscheidungen. Diese Trennung reduziert Fehlerquellen und erhöht die Validierbarkeit. Für die Interaktion werden ergänzend Tooltips, Hervorhebungen und Zeitsteuerung vorgesehen, um sowohl globale als auch detailorientierte Analyse zu ermöglichen [4, 9].

5.9 Konzeption der Benutzungsoberfläche

Die Benutzungsoberfläche wird in drei logisch getrennte Bereiche gegliedert: Eingabe und Konfiguration, Simulationssteuerung sowie Ergebnisdarstellung. Diese Struktur soll die mentale Last reduzieren und den Wechsel zwischen Experimentieren und Interpretieren erleichtern.

Die Umsetzung dieser Aufteilung ist in Abbildung 5.1 exemplarisch zu sehen. Der Genera-

tor verbindet die Eingabe der Task-Daten mit einer Vorschau und vordefinierten Szenarien.

Szenario-Generator Schliessen

Titel
Versetzt

Beschreibung
Prozesse treffen nacheinander ein. Unter einem passenden präemptiv

Seed
33

Tick-Größe
1

Klassisch Versetzt Lang Viele Chaos

SRTF RR-Quantum Strict-Priority-Starvation

MLFQ-Treppe Zufall

ID	NAME	ANKUNFT	RECHENZEIT	PRIO	FARBE
A	A	0	6	2	—
B	B	2	4	1	—
C	C	4	3	4	—
D	D	5	2	2	—

+ Prozess

Szenario anwenden

Versetzt VORSCHAU

Miniatur aus den aktuellen Prozessdaten

	0	2	4	6	8
A	0 → 6				
B	2 → 6				
C	4 → 7				
D	5 → 7				

Figure 5.1: Szenario-Generator zur Definition von Task-Daten und Simulationsparametern.

Zentrale Interaktionsprinzipien sind Konsistenz, Sichtbarkeit des Systemzustands und fehlertolerante Bedienung. Aktionen müssen eindeutig rückgemeldet werden, insbesondere beim Wechsel zwischen pausiertem und laufendem Zustand. Modale Dialoge und Menüs sind mit klarem Fokusverhalten und semantischer Beschriftung auszustatten, damit die Anwendung auch mit assistiven Technologien nutzbar bleibt [14]. Insgesamt bildet die Oberflächenkonzeption damit die operative Umsetzung der in Kapitel 3 und Kapitel 4 abgeleiteten didaktischen und HCI-Anforderungen.

6 Implementierung

Dieses Kapitel beschreibt die konkrete technische Umsetzung auf Basis der zuvor definierten Anforderungen und der konzeptionellen Architektur. Im Fokus stehen die Trennung zwischen Simulationskern und Darstellung, die typischere Modellierung der Scheduling-Domäne sowie die Umsetzung interaktiver Analysefunktionen.

6.1 Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als rein clientseitige Webanwendung umgesetzt. Der gewählte Technologie-Stack besteht aus Vue 3, TypeScript und Vite [13, 11]. Diese Kombination unterstützt eine schnelle Entwicklungsiteration und bietet zugleich eine klare Struktur für komponentenbasierte Oberflächen und typischere Fachlogik.

Vue 3 wird für die komponentenorientierte UI-Schicht eingesetzt. TypeScript dient der expliziten Modellierung von Tasks, Ereignissen, Simulationsergebnissen und Metriken. Vite übernimmt Build- und Entwicklungsworkflow inklusive schneller Aktualisierung während der Entwicklung. Für automatisierte Tests wird Vitest verwendet [12]. Animationsbezogene UI-Effekte werden mit GSAP realisiert, ohne in die Simulationslogik einzugreifen [3]. Zusätzlich ist die Oberfläche mit einem Willkommensdialog, einem Szenario-Generator und einer strukturierten Algorithmusauswahl erweitert, die den Nutzungsfluss für Lehr- und Vergleichszwecke deutlich verbessern.

6.2 Architektur der Anwendung

Die Implementierung folgt einer klaren Schichtenlogik:

- **Domänenkern:** deterministische Simulationslogik für Scheduling-Verfahren.
- **Anwendungslogik:** Verwaltung von Szenarien, Runs und Vergleichsdaten.
- **Darstellungsschicht:** Visualisierung der Ergebnisse und Interaktion mit Nutzenden.

Zentral ist dabei die Einbahnbeziehung vom Kern zur Darstellung: Die UI berechnet keine Scheduling-Entscheidungen, sondern rendert die vom Kern gelieferten Segmente, Ereignisse, Snapshots und Metriken. Dadurch wird verhindert, dass Visualisierung und Berechnungslogik auseinanderlaufen.

Ein weiterer Architekturpunkt ist die bewusste Entkopplung von Simulationszustand und Wiedergabesteuerung. Die Simulation liefert einen konsistenten Ergebnisdatensatz; die Wiedergabelogik steuert nur, welcher Zustand zu einem Zeitpunkt angezeigt wird. Das verbessert Testbarkeit und Wiederholbarkeit von Experimenten.

6.3 Modellierung der Scheduling-Verfahren

Die fachliche Modellierung basiert auf einem gemeinsamen Task-Modell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität und dynamischem Ausführungszustand. Darauf aufbauend werden die einzelnen Scheduling-Verfahren als Strategien innerhalb desselben Simulationsrahmens umgesetzt.

Round Robin wird über ein Zeitquantum modelliert, Strict Priority über prioritätsbasierte Auswahl inklusive Präemption bei höherrangigen Ankünften, LCFS über last-in-first-out-Mechanik und MLFQ über mehrere Warteschlangenebenen mit regelbasierter Herabstufung. Damit wird die in der Literatur beschriebene algorithmische Charakteristik im Kern einheitlich und vergleichbar operationalisiert [8, 10, 7].

Wichtig für die Vergleichbarkeit ist, dass alle Verfahren über dieselbe Tick- und Ereignislogik laufen. Unterschiede im Ergebnis ergeben sich damit aus dem jeweiligen Scheduling-Verhalten und nicht aus abweichenden Ausführungsmodellen.

6.4 Umsetzung der Simulation

Die Simulation ist als diskreter, deterministischer Ablauf umgesetzt. Zu jedem Simulationsschritt werden neue Ankünfte verarbeitet, Auswahlregeln angewendet und gegebenenfalls Präemptionen oder Kontextwechsel protokolliert. Dadurch entsteht ein vollständiger Verlauf, der sowohl für die visuelle Darstellung als auch für die Metrikberechnung genutzt wird.

Konkret erzeugt der Kern mehrere Ergebnisformen:

- **Ereignislog:** chronologische Ereignisse wie Dispatch, Start, Präemption und Abschluss.
- **Segmente:** Zeitabschnitte für die Gantt-Darstellung.
- **Snapshots:** zustandsbasierte Zwischenbilder für Wiedergabe und Zeitsprünge.
- **Metriken:** aggregierte Kenngrößen pro Simulationslauf.

Diese Mehrfachausgabe aus einem Kernmodell entspricht den Prinzipien diskreter Ereignissimulation und verbessert die Nachvollziehbarkeit der Laufzeitdynamik [1].

6.5 Umsetzung der Visualisierung

Die Visualisierung erfolgt auf Basis von SVG-Komponenten. Diese Wahl ermöglicht präzise Positionierung auf der Zeitachse, saubere Segmentdarstellung und semantisch adressierbare Interaktionselemente. Jeder Balken in der Gantt-Ansicht repräsentiert einen konkreten Ausführungsabschnitt eines Tasks. Die Anwendung stellt die Simulation in zwei Hauptansichten dar: der Fokusansicht und der Vergleichsübersicht.

Die Oberfläche reagiert abhängig vom aktuellen Anwendungszustand. Beim initialen Aufruf stehen noch keine Simulationsdaten zur Verfügung, weshalb die Fokusansicht und die Kennzahlen mit einem erklärenden Leerzustand dargestellt werden (Abbildung 6.1). Nach dem Laden eines Szenarios werden die Task-Daten bereits in der Zeitachse sichtbar, während die Ausführungs- und Kennzahlenbereiche bis zur Anwendung eines Algorithmus deaktiviert bleiben (Abbildung 6.2). Erst nach diesem Schritt wird die eigentliche Simulation erzeugt und kann abgespielt werden.

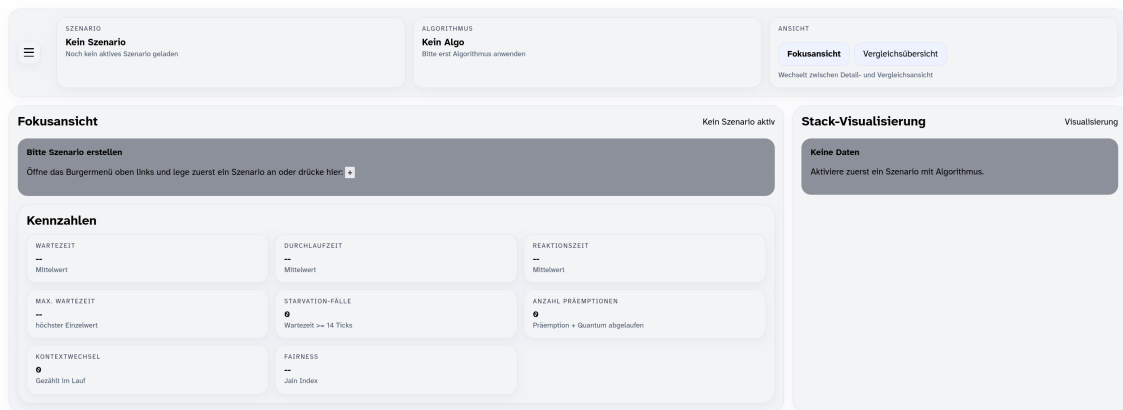


Figure 6.1: Ausgangszustand ohne Szenario und Algorithmus.

Die Darstellung nutzt die vom Simulationskern gelieferten Segment- und Snapshot-Daten direkt. Dadurch lassen sich sowohl statische Vergleiche als auch dynamische Wiedergabevarianten umsetzen. Interaktive Hilfen wie Tooltips und Hervorhebungen unterstützen die detaillierte Analyse einzelner Entscheidungen und Kontextwechsel.

6.5.1 Fokusansicht

Die Fokusansicht zeigt einen einzelnen, ausgewählten Lauf. Dieser Lauf entsteht aus einem ausgewählten Algorithmus, der auf ein konkretes Szenario angewendet wurde. Die Ansicht verbindet drei Bestandteile: das Gantt-Modell, die Stack-Visualisierung und die Kenn-

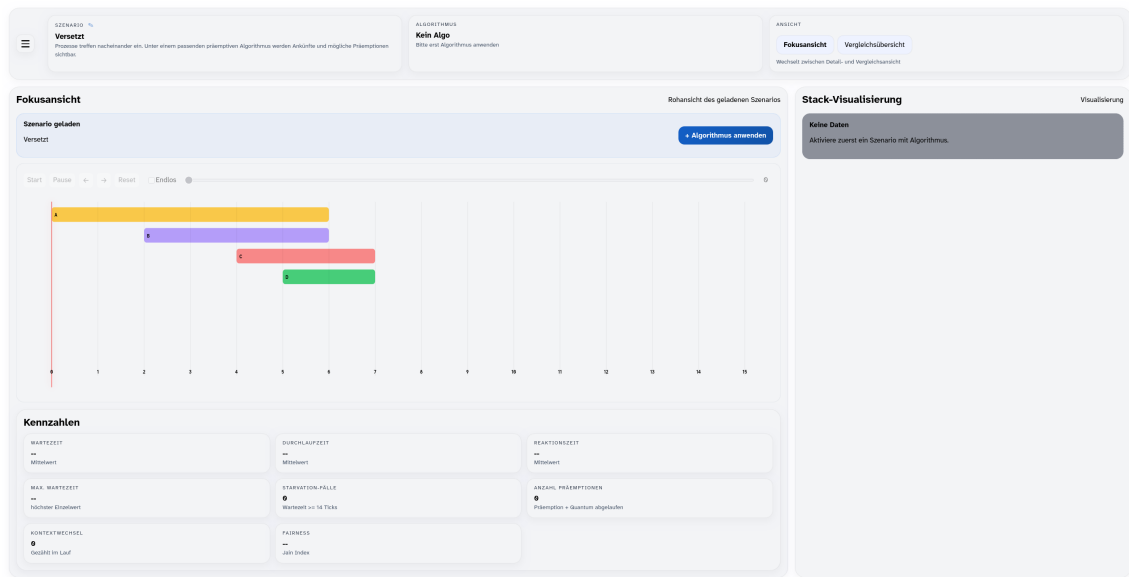


Figure 6.2: Zustand nach dem Laden eines Szenarios ohne angewendeten Algorithmus.

zahlen. Abbildung 6.3 zeigt diese Bestandteile in ihrer gemeinsamen Anordnung.

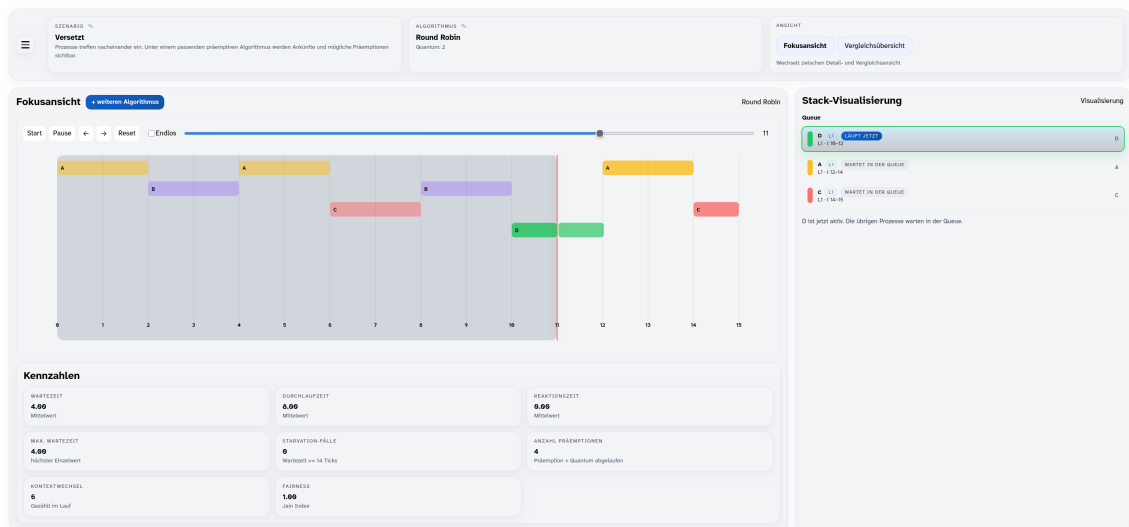


Figure 6.3: Fokusansicht mit Gantt-Zeitachse, Stack-Visualisierung und Kennzahlen.

Die Gantt-Zeitachse bildet den zeitlichen Ablauf des ausgewählten Laufs ab. Die Segmente zeigen, welcher Task in welchem Zeitintervall ausgeführt wird. An den Segmentgrenzen lassen sich insbesondere Präemptionen, abgelaufene Zeitquanten und Wechsel zwischen Tasks nachvollziehen (Abbildung 6.4).

Die Fokusansicht verfügt über einen Wiedergabemodus. Die Wiedergabe bewegt sich entlang der vom Simulationskern erzeugten Snapshots. Der aktuelle Simulationszeitpunkt wird dabei in der Gantt-Ansicht markiert. Die Stack-Visualisierung orientiert sich an

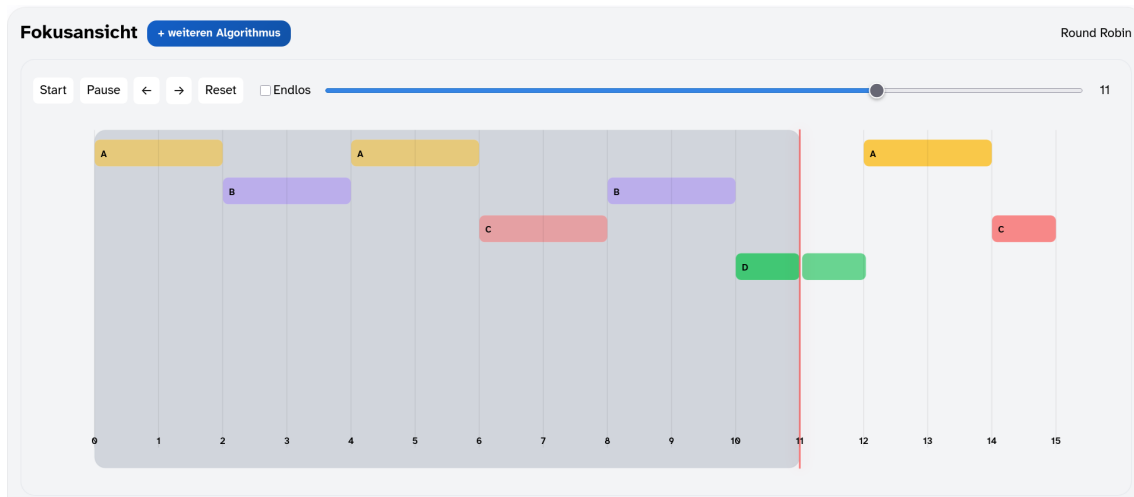


Figure 6.4: Detailansicht der Gantt-Zeitachse mit Ausführungssegmenten und markiertem Simulationszeitpunkt.

demselben Zeitpunkt und zeigt den aktiven Task sowie die wartenden Tasks mit ihrem jeweiligen Status (Abbildung 6.5). Dadurch kann die Auswahlentscheidung des Algorithmus unmittelbar mit der sichtbaren Queue-Situation verknüpft werden.



Figure 6.5: Detailansicht der Stack-Visualisierung mit aktivem und wartenden Task.

Die Kennzahlen orientieren sich ebenfalls am aktuellen Simulationslauf und werden aus dessen Berechnungsergebnis abgeleitet. Neben Warte-, Durchlauf- und Reaktionszeit werden maximale Wartezeit, Starvation-Fälle, Präemptionen, Kontextwechsel und der Fairness-Index ausgewiesen (Abbildung 6.6). Während des Wiedergabemodus ändern sich die laufbezogenen Zustandsinformationen synchron zur Wiedergabe; die abschließenden Kennzahlen fassen den vollständig berechneten Lauf zusammen.



Figure 6.6: Detailansicht der berechneten Kennzahlen eines Simulationslaufs.

6.5.2 Vergleichsübersicht

Die Vergleichsübersicht dient der kompakten Gegenüberstellung aller bisher erstellten Runs des ausgewählten Szenarios. Die Gesamtansicht verbindet die nebeneinander angeordneten Läufe mit der darunterliegenden Bewertungsmatrix (Abbildung 6.7).

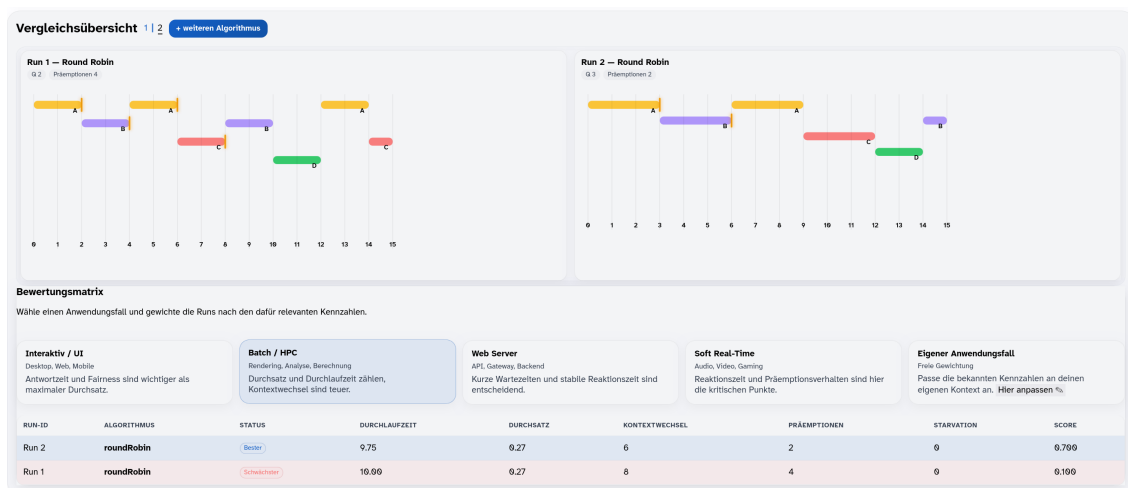


Figure 6.7: Gesamtübersicht der Vergleichsansicht mit Mini-Gantt und Bewertungsmatrix.

Jeder Run wird als Mini-Gantt dargestellt und eindeutig mit seinem verwendeten Algorithmus und seinen relevanten Parametern gekennzeichnet. Die Runs werden nebeneinander angeordnet, damit Unterschiede in Ausführungsreihenfolge, Durchlauf und Unterbrechungen schnell erfasst werden können (Abbildung 6.8).

Im Gegensatz zur Fokusansicht besitzt die Vergleichsübersicht keinen Wiedergabemodus. Sie zeigt die berechneten Verläufe statisch und konzentriert sich auf den direkten Vergleich mehrerer Runs. Präemptionen werden durch einen orangefarbenen Strich am Ende des betroffenen Ausführungssegments markiert. Dadurch sind Unterbrechungen auch in der verkleinerten Mini-Gantt-Darstellung sichtbar, ohne dass die einzelnen Runs schrittweise abgespielt werden müssen.

Die Vergleichsübersicht wird durch die Bewertungsmatrix ergänzt. Diese ordnet die

vorhandenen Runs anhand ausgewählter Kennzahlen und eines Anwendungskontexts ein. Die Matrix ist damit nicht selbst Teil der Simulationsberechnung, sondern eine zusätzliche Vergleichsschicht für die strukturierte Interpretation der bereits berechneten Ergebnisse.

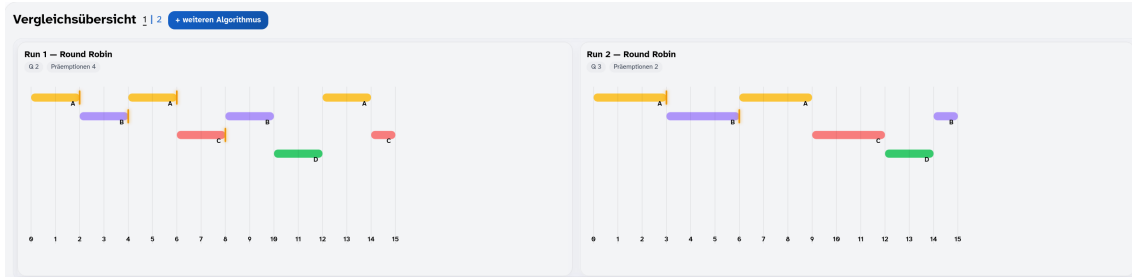


Figure 6.8: Vergleichsübersicht mit nebeneinander dargestellten Mini-Gantts mehrerer Runs.

Die Bewertungsmatrix wählt abhängig vom Anwendungskontext unterschiedliche Gewichungen für die vorhandenen Kennzahlen. Im dargestellten Beispiel wird ein Batch-/HPC-Kontext ausgewählt. Die Tabelle ordnet die Runs anschließend anhand von Durchlaufzeit, Durchsatz, Kontextwechseln, Präemptionen und Starvation und kennzeichnet den jeweils besser bewerteten Lauf.

Bewertungsmatrix
Wähle einen Anwendungsfall und gewichte die Runs nach den dafür relevanten Kennzahlen.

Interaktiv / UI
Desktop, Web, Mobile
Antwortzeit und Fairness sind wichtiger als maximaler Durchsatz.

Batch / HPC
Rendering, Analyse, Berechnung
Durchsatz und Durchlaufzeit zählen, Kontextwechsel sind teuer.

Web Server
API, Gateway, Backend
Kurze Wartezeiten und stabile Reaktionszeit sind entscheidend.

Soft Real-Time
Audio, Video, Gaming
Reaktionszeit und Präentionsverhalten sind hier die kritischen Punkte.

Eigener Anwendungsfall
Freie Gewichtung
Passe die bekannten Kennzahlen an deinen eigenen Kontext an. [Hier anpassen %](#)

RUN-ID	ALGORITHMUS	STATUS	DURCHLAUFZEIT	DURCHSATZ	KONTEXTWECHSEL	PRÄEMPTIONEN	STARVATION	SCORE
Run 2	roundRobin	Besser	9.75	0.27	6	2	0	0.700
Run 1	roundRobin	Schlechter	10.00	0.27	8	4	0	0.100

Figure 6.9: Bewertungsmatrix zur gewichteten Einordnung mehrerer Simulationsläufe.

Für visuelle Übergänge und Aufmerksamkeitssteuerung werden selektiv Animationen eingesetzt. Diese sind jedoch bewusst dekorativ entkoppelt und haben keinen Einfluss auf den fachlichen Simulationszustand [3].

6.6 Umsetzung der Interaktionsmechanismen

Die Interaktionsmechanismen orientieren sich am Lernworkflow: Szenario definieren, Algorithmus auswählen, Ablauf steuern, Ergebnis interpretieren. Daraus ergeben sich konkrete Steuerfunktionen für Start, Pause, Schrittbetrieb, Geschwindigkeitsanpassung und Zeitsprung.

Die Wiedergabelogik ist als separater Anwendungsbaustein realisiert. Sie konsumiert vorberechnete Snapshots und steuert ausschliesslich die Darstellung des aktuellen Zeitpunkts. Dadurch bleiben Interaktionen reaktionsschnell, ohne den Simulationskern neu auszuführen.

Ergänzend wurde auf robuste Bedienbarkeit geachtet: klare Statusrückmeldungen, nachvollziehbares Fokusverhalten und semantisch benannte UI-Elemente. Damit werden die in Kapitel 4 formulierten HCI-Anforderungen in konkrete Implementierungsmechanismen übersetzt [14, 2]. extbfSzenario-Management und Import/Export

Ein weiteres wesentliches Element der Umsetzung ist das Szenario-Management, das die Verwaltung, Wiederverwendung und Weitergabe von Simulationseingaben ermöglicht. Die Anwendung erlaubt es, neue Szenarien zu erzeugen, gespeicherte Konfigurationen zu verwalten und bereits bestehende Fälle als JSON-Datei zu exportieren. Der Export umfasst dabei nicht nur die Prozessdaten, sondern ebenfalls die zugehörigen Simulationseinstellungen, die angelegten Runs, die Parameter der verwendeten Scheduling-Strategien sowie den aktuell aktiven Lauf. Dadurch bleibt der vollständige Zustand des Szenarios nach einem Import konsistent und wiederherstellbar, ohne dass relevante Informationen verloren gehen oder manuell rekonstruiert werden müssen.

Dieser Mechanismus ist nicht nur als technischer Komfort zu verstehen, sondern leistet auch einen Beitrag zur fachlichen Wiederverwendbarkeit und didaktischen Praktikabilität der Anwendung. Lehrende können Standard-Szenarien erstellen und weitergeben, Nutzerinnen und Nutzer können eigene Konfigurationen speichern und zwischen verschiedenen Analysephasen weiterverwenden, und Vergleichsexperimente können auf identischem Ausgangszustand reproduziert werden. Die Speicherung in einem offenen, textbasierten Format wie JSON erhöht zudem die Austauschbarkeit der Daten und reduziert die Abhängigkeit von einer spezifischen Laufzeitumgebung.

In der Architektur des Systems ist das Szenario-Management als Teil der Anwendungslogik umgesetzt und greift auf dieselben Normalisierungs- und Validierungsmechanismen zurück wie die Erstellung neuer Szenarien. Dadurch wird sichergestellt, dass importierte Daten denselben strukturellen Regeln unterliegen wie direkt im Generator erzeugte Daten. Die so realisierte Zustandsintegrität ist für die fachliche Nachvollziehbarkeit und die Vergleichbarkeit von Läufen von zentraler Bedeutung, da sie verhindert, dass visuelle Darstellung und Simulationskontext auseinanderlaufen.

6.7 Besondere technische Herausforderungen

Eine zentrale Herausforderung bestand in der Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Einerseits musste die fachliche Korrektheit bei Präemption, Kontextwechsel und MLFQ-Regelwerk sichergestellt werden, andererseits sollte die Anwendung unmittelbar auf Eingaben reagieren.

Ein zweiter Schwerpunkt war die Sicherung von Reproduzierbarkeit. Dazu wurden deterministische Ausführungspfade und testbare Schnittstellen priorisiert. Automatisierte Tests prüfen sowohl Kernlogik als auch zentrale Integrationsfälle, wodurch Regressionen früh sichtbar werden [12].

Schliesslich ergab sich ein Trade-off zwischen Detailtiefe und Ressourcenverbrauch: Dichte Snapshot-Folgen verbessern Analyse und Wiedergabe, erhöhen jedoch Speicherbedarf. Diese Spannung wurde durch parametrisierbare Snapshot-Intervalle adressiert, sodass zwischen Genauigkeit und Laufzeitverhalten situationsgerecht balanciert werden kann.

6.8 Teststrategie und Qualitätssicherung

Die Qualitätssicherung konzentriert sich auf den Simulationskern und die Anwendungslogik. Die Tests werden mit Vitest ausgeführt und prüfen sowohl isolierte Simulationseigenschaften als auch zentrale Integrationspfade der Szenarioverwaltung [12]. Ein Schwerpunkt liegt auf der Deterministik: Wird ein identisches Szenario zweimal ausgeführt, müssen Gesamtdauer, Prozessabschluss, Ereignisfolge und Ausführungssegmente übereinstimmen.

Darüber hinaus werden die fachlich relevanten Verhaltensweisen der Verfahren mit kleinen, gezielt konstruierten Prozessmengen geprüft. Dazu gehören der Quantum-Ablauf und die Präemption bei Round Robin, die Präemption bei einer neuen Ankunft im präemptiven LCFS-Verfahren sowie die Herabstufung eines Prozesses nach Ablauf seines Quantums in der MLFQ. Ergänzend prüft die Eingabevalidierung, ob ungültige oder unvollständige Szenariodaten normalisiert werden und ob die erzeugten Zeitsegmente gültige Grenzen besitzen.

Die Tests der Anwendungslogik ergänzen diese Kernprüfungen um die Verwaltung von Szenarien und Runs. Dabei wird unter anderem kontrolliert, ob aus einem Szenario und einem ausgewählten Run ein vollständiges Simulationsszenario erzeugt wird, ob Prozessdaten dabei kopiert werden und ob Änderungen an Szenario, Parametern oder aktivem Run zu einer veränderten Darstellung führen.

Zum Zeitpunkt der Erstellung dieser Arbeit umfasst die automatisierte Testsuite elf Testdateien mit insgesamt 34 erfolgreichen Tests. Die Tests sichern die technische Konsistenz der Implementierung ab, ersetzen jedoch keine fachliche Prüfung aller denkbaren Prozesskombinationen und keine empirische Untersuchung der Nutzung oder Lernwirkung. Die in Kapitel 7 verwendeten Evaluationsszenarien dienen daher zusätzlich als nachvollziehbare Prüf- und Vergleichsfälle.

7 Ergebnis und Produktpräsentation

Dieses Kapitel stellt das entwickelte Endprodukt vor und dokumentiert die zentralen Funktionen der Anwendung in ihrer fertigen Form. Im Unterschied zu einer klassischen empirischen Evaluation steht hier weniger die statistische Messung im Mittelpunkt, sondern die fachlich belastbare und visuell nachvollziehbare Produktwirkung. Ziel ist es, das entstandene Werkzeug als nutzbares Ergebnis eines engineering-orientierten Entwicklungsprozesses zu präsentieren und damit die praktische Umsetzung der fachlichen Anforderungen sichtbar zu machen.

7.1 Entwickeltes Endprodukt

Die entwickelte Webanwendung stellt einen interaktiven Scheduling-Visualizer dar, der verschiedene präemptive CPU-Scheduling-Verfahren in einem gemeinsamen Lern- und Analysekontext verknüpft. Die Anwendung erlaubt es, Tasks mit Ankunftszeit, Burst-Zeit, Priorität und weiteren Attributen ein Szenario zu definieren, einen Algorithmus auszuwählen und den Ablauf zeitlich sichtbar zu machen.

Das fertige Produkt verbindet damit drei zentrale Aspekte:

- technische Korrektheit der Scheduling-Logik,
- verständliche Darstellung des zeitlichen Ablaufs,
- und eine nutzerorientierte Oberfläche für Lehr- und Vergleichszwecke.

Damit erfüllt die Anwendung nicht nur die fachlichen Vorgaben eines Betriebssystem-Kapitels, sondern stellt zugleich ein tragfähiges und erweitertes Produkt dar, das im Studienkontext als didaktisches Werkzeug eingesetzt werden kann.

7.2 Produktüberblick und Nutzerfluss

Der typische Nutzerfluss beginnt mit dem Definieren eines Szenarios. Auf der Startseite sind die Systemgrundlagen, die Eingabeform und die Auswahl relevanter Task-Parameter direkt sichtbar. Nach dem Laden eines Szenarios kann der Nutzer den gewünschten Algorithmus auswählen und den Ablauf in verschiedenen Modi starten. Anschließend stehen eine Fokusansicht und eine Vergleichsübersicht zur Verfügung.

Die Anwendung wurde bewusst so aufgebaut, dass die Interaktion in logischer Reihenfolge abläuft: Eingabe, Auswahl, Ablaufsteuerung, Analyse und Vergleich. Dadurch bleibt der Nutzer bei jedem Schritt im Kontext des jeweiligen Lernziels und muss keine unnötigen Bedienhürden überwinden.

7.3 Produktpräsentation mit Screenshots

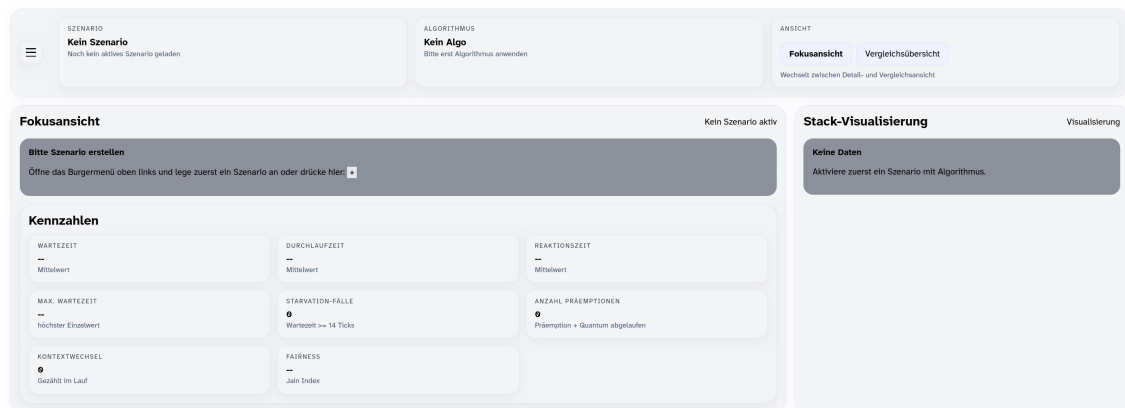


Figure 7.1: Startansicht der Anwendung mit leerem Zustand und erklärender Ausgangslage. Der Nutzer startet mit der Definition eines Szenarios und der Auswahl eines Scheduling-Verfahrens.

7.4 Umgesetzte Funktionalität

Die fertige Anwendung erfüllt die wesentlichen Anforderungen, die im Verlauf der Arbeit aus fachlicher, didaktischer und HCI-orientierter Sicht definiert wurden. Zu den wichtigsten Funktionen gehören:

- Erstellung und Bearbeitung von Task-Daten mit Ankunftszeit, Burst-Zeit und Priorität,
- interaktive Szenariogenerierung mit vordefinierten Basis-Szenarien und direkter Vorschau,
- Auswahl und Vergleich verschiedener Scheduling-Verfahren über einen klaren Algorithmusdialog,
- kontrollierte Wiedergabe mit Start, Pause, Schrittmodus und Zeitsprung,
- visuelle Darstellung als Zeitachsenmodell in Gantt-Form,

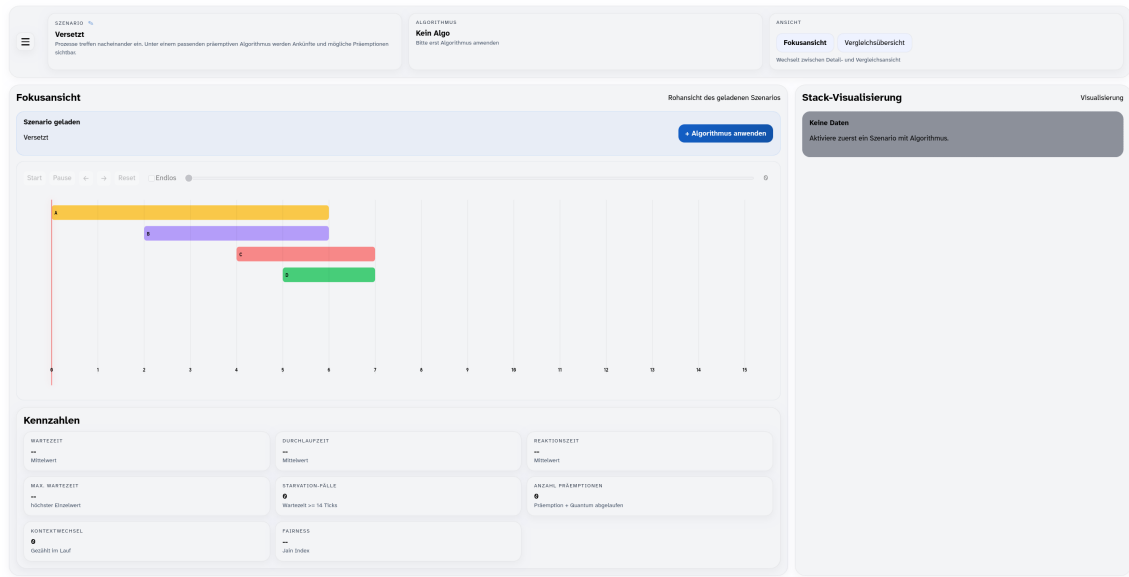


Figure 7.2: Ladezustand eines konfigurierten Szenarios. Die Task-Daten sind sichtbar, und die Simulation kann mit einer klaren Ablaufsteuerung initialisiert werden.

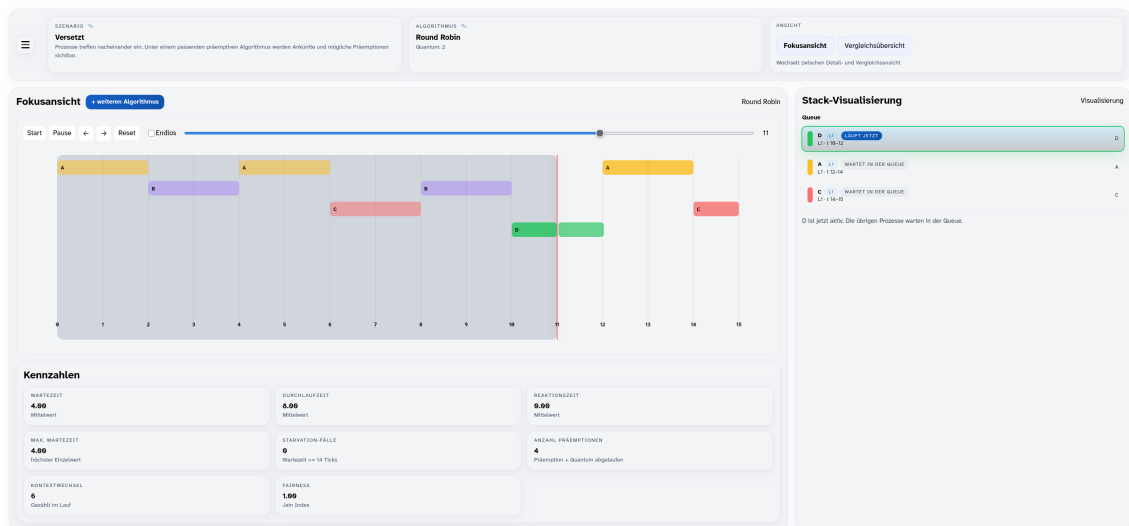


Figure 7.3: Fokusansicht der Simulation mit Gantt-Zeitachse, aktiver Task-Darstellung und Kennzahlbereich. Diese Ansicht zeigt den Ablauf des gewählten Scheduling-Verfahrens in einer verständlichen, zeitlich strukturierten Form.

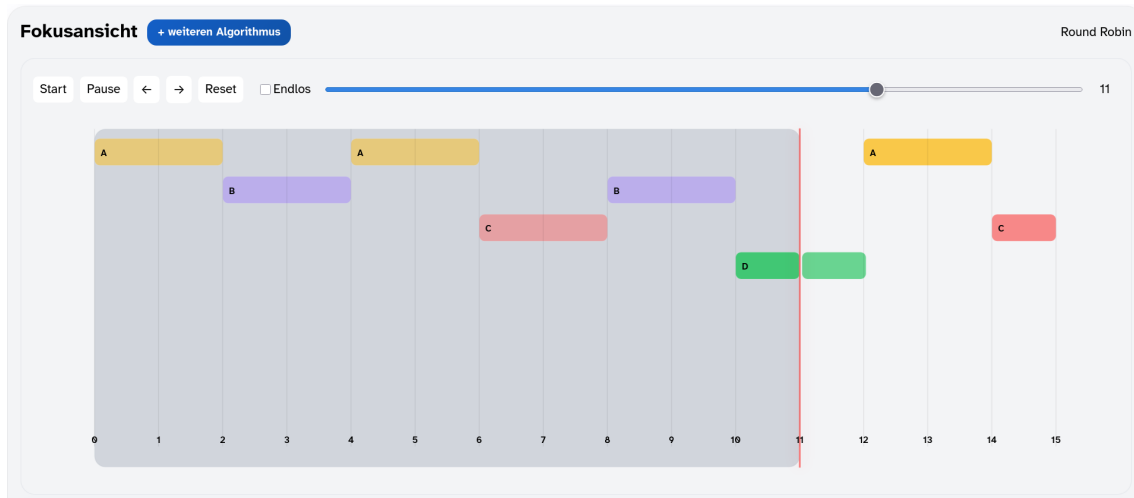


Figure 7.4: Detailansicht der Gantt-Darstellung. Einzelne Ausführungssegmente, Präemptionen und Task-Wechsel werden als konkrete Zeitabschnitte sichtbar gemacht.

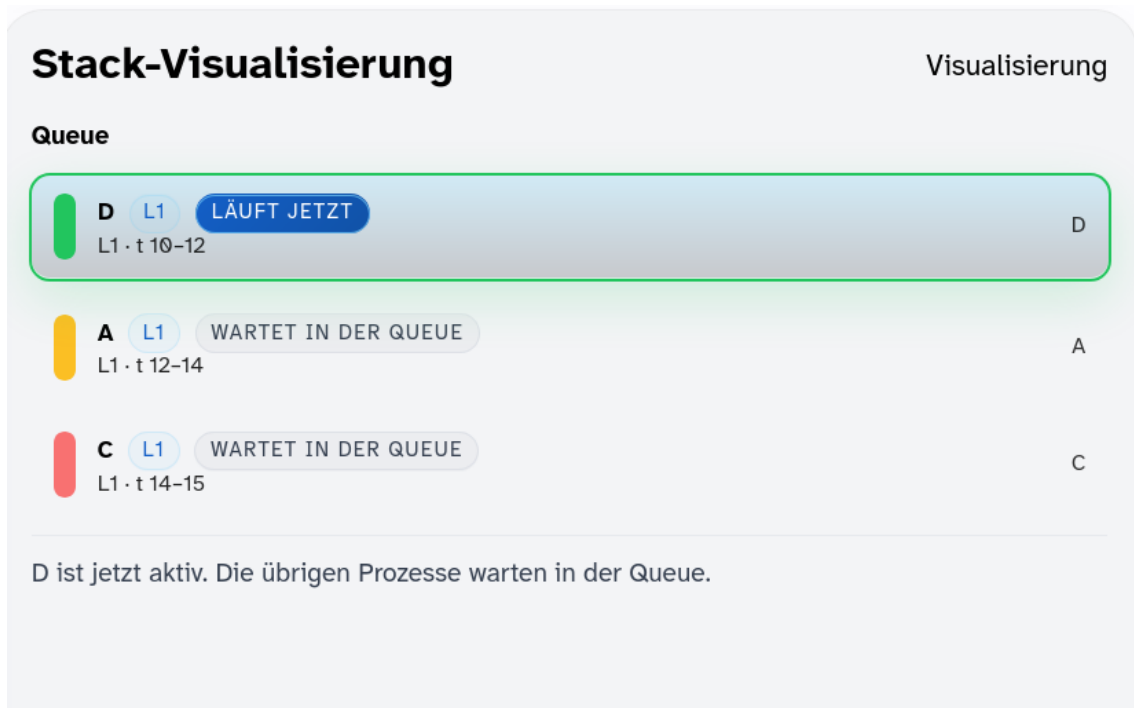


Figure 7.5: Darstellung des Task-Status in der Warteschlangenansicht. Der aktuelle Zustand und die Warteliste werden gleichzeitig präsentiert, wodurch die Auswahlentscheidung des Algorithmus nachvollziehbar wird.

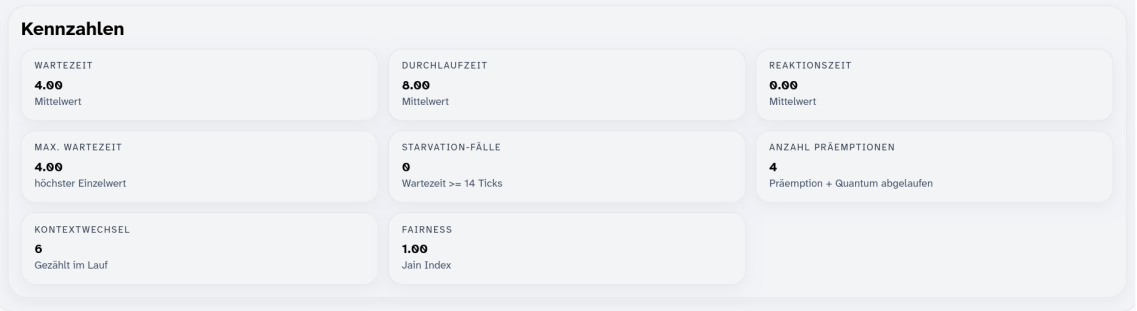


Figure 7.6: Kennzahlenbereich mit Warte-, Durchlauf- und Reaktionszeiten. Die Metriken geben eine kompakte fachliche Bewertung des Simulationslaufs wieder.

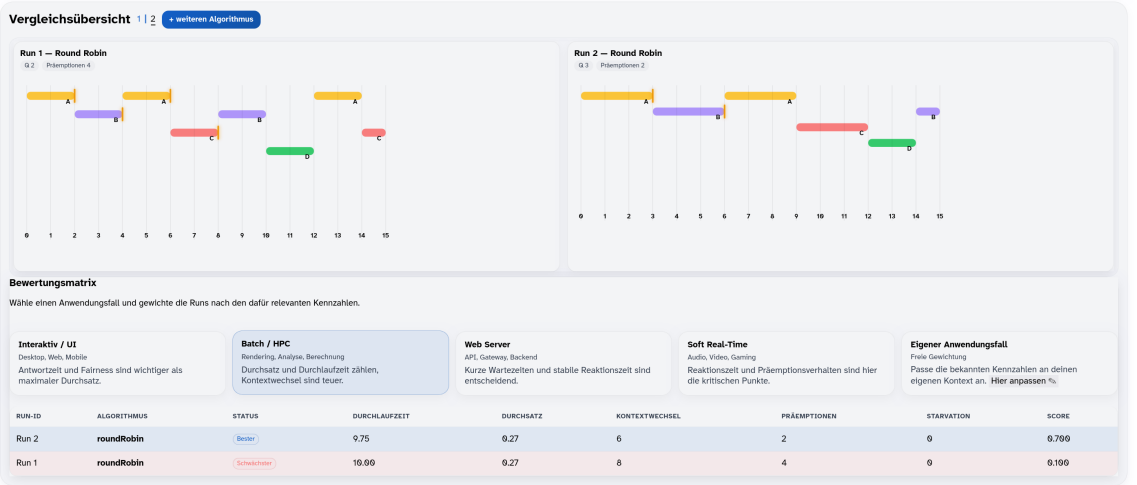


Figure 7.7: Vergleichsübersicht mit mehreren Läufen unter identischem Szenario. Die Darstellung ermöglicht den praktischen Vergleich verschiedener Scheduling-Verfahren anhand des selben Dateninputs.

- Ausgabe relevanter Kennzahlen pro Ablauf,
- Vergleich mehrerer Läufe auf identischem Szenario mit Bewertungs- und Matrixansicht.

Diese Funktionen zeigen, dass die Anwendung als echtes Produkt und nicht nur als akademische Demonstration umgesetzt wurde. Sie ermöglicht fachliche Analyse, didaktische Nachvollziehbarkeit und konkrete Produktnutzung in einem integrativen Ablauf, wobei die Benutzerführung durch Szenario-Generator, Vergleichsansichten und strukturierte Bedienpfade zusätzlich gestärkt wird.

7.5 Ergebnisbetrachtung

Das endständige Produkt erfüllt die ursprüngliche Zielsetzung der Arbeit: Es macht die Dynamik präemptiver Scheduling-Verfahren anschaulich, verständlich und vergleichbar. Die wichtigsten fachlichen Effekte, etwa Präemption, Kontextwechsel, Prioritätsverzug und Wartezeiten, werden nicht nur als Zahlenwerte gezeigt, sondern auch im zeitlichen Ablauf sichtbar gemacht.

Diese Kombination aus technischer Korrektheit und visueller Transparenz ist für das Produkt besonders wertvoll. Der Nutzer sieht nicht nur das Ergebnis eines Algorithmus, sondern auch die Entscheidungspfade, die zu diesem Ergebnis geführt haben. Dadurch wird die Anwendung zu einem nutzbaren Werkzeug für Lehre, Demonstration und fachliche Erörterung.

Das Produkt zeigt außerdem, dass die Arbeit eine konkrete technische Leistung erbracht hat: Es wurde eine vollständig nutzbare, interaktive Anwendung entwickelt, die fachlich korrekte Simulation, verständliche Visualisierung und eine klare, strukturiert gestaltete Benutzerführung verbindet. Gerade diese Kombination ist im Studiengang als engineering-orientierte Leistung besonders aussagekräftig, weil sie den Übergang von abstrakter Theorie zu konkreter Softwarelösung sichtbar macht.

Die Produktpräsentation zeigt zugleich, dass die Bachelorarbeit nicht nur ein theoretisches Dokument, sondern ein konkret umgesetztes Softwaresystem ist. Diese Perspektive passt zur engineering-orientierten Ausrichtung der Arbeit und entspricht zugleich dem fachlichen Anspruch des Studiengangs, in dem methodische Genauigkeit, technische Umsetzung und verständlicher Produktaufbau gleichwertig berücksichtigt werden müssen.

8 Fazit und Ausblick

Die vorliegende Arbeit zeigt die erfolgreiche Umsetzung eines engineering-orientierten Softwaresystems zur Visualisierung und Analyse von CPU-Scheduling-Verfahren. Ausgangspunkt war die fachliche Herausforderung, zeitliche Ablaufentscheidungen in Betriebssystemen verständlich zu machen. Das Ergebnis ist ein funktionierendes Produkt, das in der Lage ist, verschiedene präemptive Verfahren in einer konsistenten, interaktiven Umgebung darzustellen.

8.1 Zusammenfassung der Ergebnisse

Im Verlauf der Arbeit wurde eine Webanwendung entwickelt, die das fachliche Wissen aus der Betriebssystemtheorie mit den Anforderungen aus Software Engineering und Human-Computer Interaction verbindet. Die Architektur trennt die fachliche Simulationslogik von der Benutzeroberfläche, wodurch die Anwendung modular, nachvollziehbar und erweiterbar bleibt. Die Oberfläche selbst stellt den Ablauf in einer klaren, didaktischen Form dar und unterstützt den Nutzer dabei, Entscheidungen, Unterbrechungen und Zeitverläufe direkt zu verstehen.

Ein zentraler Erfolg der Arbeit liegt darin, dass das fertige Produkt nicht nur die theoretischen Verfahren visualisiert, sondern auch ihren praktischen Effekt im Ablauf sichtbar macht. Durch die Kombination aus Gantt-Darstellung, Kennzahlen und Vergleichsübersicht entsteht ein Werkzeug, das nicht nur Informationen zeigt, sondern den Lern- und Analyseprozess aktiv begleitet.

8.2 Reflexion über die gestellten Anforderungen

Die in der Arbeit definierten Anforderungen wurden in einem durchgängigen Produktentwicklungsprozess umgesetzt. Dabei war die Trennung zwischen fachlicher Korrektheit und visueller Darstellung besonders wichtig. Die Anwendung zeigt nicht nur das Endergebnis eines Scheduling-Laufs, sondern macht auch die logischen Entscheidungen der einzelnen Verfahren im Zeitverlauf sichtbar.

Diese Haltung entspricht dem Anspruch einer engineering-orientierten Bachelorarbeit: Es geht nicht allein um die theoretische Beschreibung eines Algorithmus, sondern um die Gestaltung eines nachvollziehbaren und nutzbaren Systems, das fachlich korrekt, technisch

sauber und für den didaktischen Kontext relevant ist.

8.3 Ergebnis und praktische Wirkung

Die vorliegende Arbeit endet mit einem klaren technischen Ergebnis: Es wurde ein funktionierendes, interaktives und didaktisch nutzbare Produkt entwickelt, das die fachlichen Inhalte des CPU-Scheduling in eine verständliche Arbeitsumgebung übersetzt. Die Anwendung erfüllt damit nicht nur eine demonstrative Funktion, sondern besitzt einen unmittelbaren praktischen Mehrwert für Lehrende und Lernende.

Die praktische Wirkung entsteht vor allem durch die Verbindung von fachlicher Korrektheit und leicht verständlicher Bedienung. Nutzerinnen und Nutzer können Szenarien verändern, Tasks beobachten und die Auswirkungen verschiedener Scheduling-Verfahren unmittelbar nachvollziehen. Dadurch wird die Theorie in eine nachvollziehbare, sichtbare und experimentierbare Form überführt.

8.4 Ausblick auf Weiterentwicklung

Das Produkt bildet eine solide Grundlage für weitere Erweiterungen. Zukünftige Entwicklungen könnten zusätzliche Scheduling-Verfahren, tiefergehende Parameterstudien, fokussierte Aufgabenmodi für Lernende oder eine noch stärker auf Accessibility ausgerichtete Oberfläche umfassen.

Darüber hinaus wäre eine Weiterentwicklung durch zusätzliche Erklärtexpte, interaktive Hinweise und größere Vergleichsszenarien sinnvoll, um die Lehrwirkung und den produktiven Einsatz im Studienkontext weiter zu stärken. Auch eine systematische Integration in Lehrveranstaltungen könnte zeigen, inwiefern die Anwendung als didaktisches Werkzeug im realen Einsatz genutzt werden kann.

Das zentrale Ergebnis bleibt jedoch unverändert: Es wurde ein vollständiges, funktionierendes und fachlich belastbares Produkt geschaffen, das die Prinzipien des CPU-Scheduling verständlich und praktisch erfahrbar macht.

Bibliography

- [1] Jerry Banks, John S. Carson II, Barry L. Nelson, and David M. Nicol. *Discrete-Event System Simulation*. Pearson, 5 edition, 2010. Methodik zur diskreten Ereignissimulation, Validierung, Reproduzierbarkeit.
- [2] John Brooke. Sus: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, 1996. Originalquelle zur System Usability Scale; oft zitiert fuer Usability-Evaluierungen.
- [3] GreenSock. Gsap documentation. Library fuer zeitbasierte Animationen; verwendet in diesem Projekt fuer Timeline-Animationen.
- [4] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3):259–290, 2002. Definitive meta-study: Wichtig fuer Argumente zur Lehrwirksamkeit von Visualisierungen.
- [5] L. W. C. Meing and D. R. Tandingan. Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms. *Journal of Information Systems Engineering and Management (JISEM)*, 2025. Studie zu MLFQ-Kombinationen und dynamischer Zeitscheibenwahl.
- [6] T. L. Naps and Others. Exploring the role of visualization and engagement in computer science education. In *SIGCSE Technical Symposium on Computer Science Education (proceedings)*, 2002. Grundlagen zur Engagement-Taxonomie (viewing vs. interacting vs. constructing).
- [7] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 4 edition, 2016. Formale Algorithmenbetrachtung und Performance-Metriken.
- [8] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018. Standardwerk: Scheduling-Definitionen und Metriken.
- [9] John T. Stasko, John Domingue, Marc H. Brown, and Bruce A. Price, editors. *Software Visualization: Programming as a Multimedia Experience*. MIT Press, 1998. Designprinzipien und Taxonomien fuer Software/Algorithmus-Visualisierung.

- [10] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014. Konzeptuelle Tiefe zu Praemption, Kontextwechseln und Scheduler-Design.
- [11] Vite. Vite - build tool. Dev/Build Tool im Projekt.
- [12] Vitest Team. Vitest - unit testing for vite. Test-Runner im Projekt.
- [13] Vue.js Core Team. Vue 3 documentation (composition api). Framework des Projekts (SFCs, Composition API).
- [14] W3C. Web content accessibility guidelines (wcag) 2.1, 2018. Zugaenglichkeitsanforderungen (ARIA, Tastaturzugriff, Kontrast, etc.).

9 Anhang

Der Anhang enthält ergänzende Materialien, zum Beispiel zusätzliche Screenshots, Testfälle und tabellarische Auswertungen.

Die Zustandsübergänge der Benutzeroberfläche vom initialen Leerzustand bis zum geladenen Szenario sind in Kapitel 6 dokumentiert. Die zugehörigen Originaldateien liegen im Bildverzeichnis des Projekts und können bei Bedarf für eine umfangreichere Anhangsdokumentation verwendet werden.